

EQUATIONS — for spot-the-error

Base (accept keeps $s \Rightarrow V_s$; reject redraws $\Rightarrow Q$):

$$V_s = \max\{\bar{w}_s + \beta V_s, c + \beta Q\}, \quad Q = p \cdot V$$

flow ($\bar{w}_s/c$) undiscounted; β on continuation only. Finite: RHS uses V_{t-1}, Q_{t-1} ; base $V_{1,s} = \max(\bar{w}_s, c)$.

w*: accept iff $\bar{w}_s \geq w^*$, $\frac{w^*}{1-\beta} = c + \beta Q$.

OJS ($W_s = \sum p_{s'} \max\{V_s, V_{s'}\}$, keep best):

$$V_s = \max\{\bar{w}_s + \beta[(1-\lambda)V_s + \lambda W_s], c + \beta Q\}$$

Firing (fired w.p. $\alpha \rightarrow U$): replace λW_s by αU , accept branch $V_s + \beta[(1-\alpha)V_s + \alpha U]$.

Arrival ($U = c + \beta[\lambda Q + (1-\lambda)U]$).

common errors: swapped branches ($V_s \leftrightarrow Q$); unweighted mean instead of dot(p, V); missing β ; reject using V not Q .

BASE McCALL

iterateBellman *one update* $\rightarrow (V, Q, C)$

```
function iterateBellman(V,Q,β,p,ŵ,c)
  V_acc = ŵ ./ β .* V # accept: ŵ[s]+β·V[s]
  V_rej = c ./ β .* Q # reject: c+β·Q (scalar)
  V_new = max.(V_acc, V_rej)
  Q_new = dot(p, V_new) # prob-weighted, NOT mean
  C = V_acc .>= V_rej # !accept Q=reject
  return (V=V_new, Q=Q_new, C=C)
end
```

solveBellman *VFI to fixed point*

```
function solveBellman(β,p,ŵ,c)
  V = max.(ŵ,c) # guess A (t=1 payoff)
  Q = dot(p,V); dist = 1.0; C = ŵ.>=c
  while dist > 1e-10
    V_new,Q_new,C = iterateBellman(V,Q,β,p,ŵ,c)
    dist = norm(V.- V_new, Inf) # sup-norm
    V = V_new; Q = Q_new
  end
  w_res = ŵ[findfirst(C.==1)] # lowest accepted
  return (V=V, Q=Q, C=C, w_res=w_res)
end
```

solveBellman_finite *T periods: backward induction, store every t*

```
function solveBellman_finite(β,p,ŵ,c,T)
  S = length(ŵ)
  V = zeros(T,S); Q = zeros(T); C = zeros(Int,T,S)
  V[1,:]=max.(ŵ,c) # t=1, no future
  Q[1,:]=dot(p, V[1,:]); C[1,:]=ŵ.>=c
  for t in 2:T
    r = iterateBellman(V[t-1,:], Q[t-1], β,p,ŵ,c)
    V[t,:]=r.V; Q[t]=r.Q; C[t,:]=r.C
  end
  return (V=V, Q=Q, C=C)
end
# row t = "t periods left"; RHS uses t-1
```

contraction (mod β): error $\leq \beta^n$, converges from any guess. hazard $h=\text{dot}(p,C)$; $E[\text{duration}] = 1/h$; w/ arrival λ : $1/(\lambda h)$.

Initial guess — two forms both $\rightarrow$ same V^* ; pick by what V means

A $\text{max.}(\text{w}\bar{\text{d}},c)$: when reject value sits inside V (base, UI). **B** $\text{w}\bar{\text{d}}./(1-\bar{\text{f}})$: when V = value of being employed at s & search lives in separate U (firing, OJS, arrival). **B** is better-conditioned (starts at true envelope).

ON-THE-JOB SEARCH

iterateBellman (+W) *employed keep drawing, trade up*

```
function iterateBellman(V,W,Q,β,λ,p,ŵ,c)
  V_acc = ŵ ./ β .* (1-λ).*V + λ.*W
  # keep wage w.p. (1-λ); new offer=ŵ w.p. λ
  V_rej = c ./ β .* Q # reject & search
  V_new = max.(V_acc, V_rej)
  Q_new = dot(p, V_new)
  W_new = [dot(p, max.(v, V_new)) for v in V_new]
  # W: keep current v OR take a better draw
  C = V_acc .>= V_rej
  return (V=V_new, W=W_new, Q=Q_new, C=C)
end
```

solveBellman (OJS) *iterate V, W, Q together*

```
function solveBellman(β,λ,p,ŵ,c)
  V = max.(ŵ,c); C = ŵ.>=c
  Q = dot(p,V)
  W = [dot(p, max.(v, V)) for v in V]
  dist = 1.0
  while dist > 1e-10
    V_new,W_new,Q_new,C =
      iterateBellman(V,W,Q,β,λ,p,ŵ,c)
    dist = norm(V.- V_new, Inf)
    V=W_new; W=W_new; Q=Q_new
  end
  return (V=V, C=C)
end
```

solveBellman_finite (OJS) *T periods left; carry V, W, Q each step*

```
function solveBellman_finite(β,λ,p,ŵ,c,T)
  S = length(ŵ)
  V = zeros(T,S); W = zeros(T,S)
  Q = zeros(T); C = zeros(Int,T,S)
  V[1,:]=max.(ŵ,c) # t=1 (last period)
  Q[1,:]=dot(p, V[1,:])
  W[1,:]= [dot(p, max.(v, V[1,:])) for v in V[1,:]]
  C[1,:]=ŵ.>=c
  for t in 2:T
    r = iterateBellman(V[t-1,:],W[t-1,:],Q[t-1],
      β,λ,p,ŵ,c)
    V[t,:]=r.V; W[t,:]=r.W; Q[t]=r.Q; C[t,:]=r.C
  end
  return (V=V, W=W, Q=Q, C=C)
end
```

$\lambda \uparrow \Rightarrow w^* \downarrow$: $W_s \geq Q$ always (keep better of draw vs. current), so more λ -weight makes accepting cheaper — low wage isn't a trap, trade up later; w^* can fall below c .

solveMcCallOJS *NOTES version: infinite-horizon, no W vector*

```
function solveMcCallOJS(β,p,ŵ,c)
  V = ŵ ./ (1-β) # guess B
  U = c ./ (1-β); dist = 1.0
  while dist > 1e-10
    # employed at s: keep V[s] or jump to draw i
    EV = [dot(p, max.(V, V[s])) for s in eachindex(ŵ)]
    V_new = ŵ ./ β .* EV
    U_new = c ./ β .* U # unemployed
    V_dist = max(norm(V.- V_new, Inf), abs(U - U_new))
    V = V_new; U = U_new
  end
  return (V=V, U=U, C = V .>= U)
end
```

same economics as the practice-final form, but W is folded inline into EV and unemployment is its own U rather than Q .

FIRING & ARRIVAL

iterateBellman (firing α) *mirror of OJS: α, U play λ, W*

```
function iterateBellman(V,Q,U,β,p,ŵ,c,α)
  V_acc = ŵ ./ β .* ((1-α).*V + α.*U)
  # keep job w.p. (1-α); fixed=U w.p. α
  U_new = c ./ β .* Q
  V_new = max.(V_acc, U_new)
  C = V_acc .>= U_new
  return (V=V_new, Q=dot(p,V_new),
    U=U_new, C=C)
end
```

solveBellman (firing) *iterate V and U to fixed point*

```
function solveBellman(β,p,ŵ,c,α)
  V = ŵ ./ (1-β) # guess B
  U = c ./ (1-β); C = ŵ.>=c; dist = 1.0
  while dist > 1e-10
    Q = dot(p, V)
    r = iterateBellman(V,Q,U,β,p,ŵ,c,α)
    dist = max(norm(V.- r.V, Inf), abs(U - r.U))
    V = r.V; U = r.U; C = r.C
  end
  return (V=V, U=U, C=C)
end
```

$\alpha \uparrow \Rightarrow$ accept worth less (job may end) $\Rightarrow w^* \downarrow$.

solveMcCallArrival(...;λ) *offer arrives only w.p. λ*

```
function solveMcCallArrival(β,p,ŵ,c,λ)
  V = ŵ ./ (1-β) # guess B: employed value
  U = c ./ (1-β); dist = 1.0
  while dist > 1e-10
    Q = dot(p, max.(V, U)) # E[best of draw,U]
    U_new = c ./ β .* (λ.Q + (1-λ).*U)
    # offer=Q w.p. λ; none=stay U w.p. (1-λ)
    dist = abs(U - U_new); U = U_new
  end
  return (V=V, U=U, C = V .>= U)
end
```

$\lambda \downarrow \Rightarrow V \downarrow$, but w^* unchanged (per-offer choice same); only **duration** = $1/(\lambda h)$ rises.

HW — TIME-LIMITED UI (state (k,s))

k = UI periods **left**; accept exhausts UI; after, $c \rightarrow c_0$.

$$V_{k,s} = \max\{\bar{w}_s + \beta V_{0,s}, c + \beta Q_{k-1}\} \quad (k \geq 1)$$

accept jumps to $V_{0,s}$ (not $V_{k-1,s}$); V_k off RHS $\Rightarrow$ each k = ONE update; array **row** $k+1$ = state k .

iterateBellman_withUI *one k-step: target V_0 , reject uses $Q[k-1]$*

```
function iterateBellman_withUI(V0,Q_prev,β,p,ŵ,c)
  V_acc = ŵ ./ β .* V0 # accept + continue (0,s)
  V_rej = c ./ β .* Q_prev # reject + draw at k-1
  V_k = max.(V_acc, V_rej)
  return (V=V_k, Q=dot(p,V_k),
    C=V_acc.>=V_rej)
end
```

solveTimeLimited *k=0 base solve, then march k up*

```
function solveTimeLimited(β,p,ŵ,c,c0,D)
  S = length(ŵ)
  S0 = solveBellman(β,p,ŵ,c0) # k=0 = McCall(c0)
  V = zeros(D+1,S); Q = zeros(D+1)
  C = zeros(Int,D+1,S)
  V[1,:]=s0.V; Q[1]=s0.Q; C[1,:]=s0.C
  V0 = s0.V # fixed accept-target
  for k in 1:D
    r = iterateBellman_withUI(V0,Q[k],β,p,ŵ,c)
    V[k+1,:]=r.V; Q[k+1]=r.Q; C[k+1,:]=r.C
  end
  return (V=V, Q=Q, C=C)
end
```

$w_k^* \uparrow$ in k : lots of UI $\Rightarrow$ hold out; near exhaustion $c + \beta Q_{k-1} \downarrow \Rightarrow$ accept. **hazard** $\uparrow$ toward exhaustion. **D-overlap**: w_k^* depends only on UI *left*, not original D . $c_0 = c \Rightarrow$ flat = McCall(c). **D** $\uparrow$: dur $\uparrow$, exhaust $\downarrow$, wage $\uparrow$ (UI trade-off; Baily, Chetty).

Read paths over the spell *reservation wage & hazard at each k*

```
# sol = solveTimeLimited(...); k=0..D
w_res = [v[findfirst(sol.C[k+1,:].==1)]
  for k in 0:D] # reservation-wage path
h = [dot(p, sol.C[k+1,:]) for k in 0:D] # hazard path
dur = 1 ./ h # E[duration] at each k
```

simulateSpell *one worker: draw, check policy at (k,s), accept or burn UI*

```
function simulateSpell(sol,p,ŵ,D; rng)
  k = D; cat = Categorical(p)
  for t in 1:1000
    s = rand(rng, cat)
    if sol.C[k+1,s] == 1 # policy at (k,s)
      return (duration=t, wage=w[s],
        exhausted=(k==0))
    end
    k = max(k-1, 0) # burn one UI period
  end
  return (duration=1000, wage=NaN, exhausted=true)
end
```

simulateMany *N spells, fixed seed $\rightarrow$ mean stats*

```
function simulateMany(sol,p,ŵ,D; N=50_000, seed=42)
  rng = MersenneTwister(seed) # reproducible
  dur = zeros(Int,N); wg = zeros(N); ex = falses(N)
  for i in 1:N
    o = simulateSpell(sol,p,ŵ,D; rng=rng)
    dur[i]=o.duration; wg[i]=o.wage; ex[i]=o.exhausted
  end
  return (mean_duration=mean(dur),
    frac_exhausted=mean(ex), mean_wage=mean(wg))
end
```

Comparative-statics sweep *re-solve over a param, re-read w^**

```
for c_try in (1.0, 2.0, 3.0, 4.0)
  s = solveTimeLimited(β,p,ŵ,c_try,c0,D)
  wp = [v[findfirst(s.C[k+1,:].==1)] for k in 0:D]
  # plot!(0:D, wp, label="c=$c_try")
end
```

=== RUST (1987) ENGINE REPLACEMENT

EQUATIONS — for spot-the-error

Choice-specific values (cost enters NEGATIVE):

$$v^{keep}(x) = -c(x) + \beta(F_0 V)(x)$$

$$v^{rep}(x) = -RC + \beta(F_1 V)(x)$$

keep uses F_0 (mileage rises); replace uses F_1 (resets). F_1 has identical rows $\Rightarrow F_1 V$ is one scalar.

Bellman = **log-sum-exp** (EV shocks $\Rightarrow$ *expected* max, NOT plain max):

$$V(x) = \log\left(e^{v^{keep}(x)} + e^{v^{rep}(x)}\right)$$

Logit choice prob(prob of replacing):

$$P(d=1|x) = \frac{e^{v^{rep}}}{e^{v^{keep}} + e^{v^{rep}}} = \frac{1}{1 + e^{v^{keep} - v^{rep}}}$$

$$V(x) = E[\max(v^{keep}(x) + \epsilon_{-0}, v^{replace}(x) + \epsilon_{-1})] \\ = \log\left(e^{v^{keep}(x)} + e^{v^{rep}(x)}\right)$$

common errors: $+c(x)$ (sign — cost must be $-$); plain max instead of log-sum-exp; F_0 on the replace branch (must be F_1); exp.(V) instead of exp.(v_replace) in the update.

MODEL SETUP

RustModel struct *linear cost $c = \theta_1 \cdot \bar{x}$*

```
@kdf struct RustModel
  β::Float64 = 0.95 # discount
  S::Int = 50 # mileage bins
  RC::Float64 = 20.0 # replacement cost
  θ1::Float64 = 0.04 # maint. cost param
  θ3::Vector{Float64} = [0.36,0.48,0.16] # Δ probs
  x::Vector{Float64} = Float64{0:S-1} #mileage vector
  c::Vector{Float64} = θ1 .* x # cost per state (mileage)
end
```

buildTransition(θ3,S) *F0=keep (mileage $\uparrow$), F1=replace (reset)*

```
function buildTransition(θ3, S)
  K = length(θ3)
  F0 = zeros(S,S); F1 = zeros(S,S)
  for i in 1:S, k in 1:K
    j_keep = min(i+k-1, S) # keep: i + i*(k-1)
    F0[i,j_keep] += θ3[k]
    j_rep = min(k, S) # replace: any + k
    F1[i,j_rep] += θ3[k]
  end
  return F0, F1
end
```

SOLVE (VFI)

iterateBellman *one update, stable log-sum-exp*

```
function iterateBellman(m::RustModel, V, F0, F1)
  (; β, RC, c) = m
  v_keep = -c ./ β .* (F0 * V) # -cost, F0
  v_rep = -RC ./ β .* (F1 * V) # -RC, F1
  vmax = max.(v_keep, v_rep) # stabilize
  V_new = vmax ./ log.(exp.(v_keep ./ vmax)
    + exp.(v_rep ./ vmax))
  return V_new
end
```

solveBellman *iterate to fixed point from $V=0$*

```
function solveBellman(m::RustModel, F0, F1; tol=1e-10)
  V = zeros(m.S); dist = 1.0
  while dist > tol
    V_new = iterateBellman(m, V, F0, F1)
    dist = norm(V.- V_new, Inf)
  end
  return V
end
```

solveRust(m) *build F, solve, return $V + P(\text{replace})$*

```
function solveRust(m::RustModel)
  (; β, RC, c, θ3, S) = m
  F0, F1 = buildTransition(θ3, S)
  V = solveBellman(m, F0, F1)
  v_keep = -c ./ β .* (F0 * V)
  v_rep = -RC ./ β .* (F1 * V)
  P_rep = 1 ./ (1. + exp.(v_keep ./ v_rep)) # logit
  return (V=V, P=P_rep, v_keep=v_keep, v_replace=v_rep)
end
```

$V \downarrow$ in mileage (higher costs ahead). **RC**↑: replacing dearer $\Rightarrow P(rep) \uparrow$, S-curve shifts right. $\theta_1 \uparrow$: keeping dearer $\Rightarrow P(rep) \uparrow$. inner VFI iters $\approx \log(tol)/\log \beta$.

SIMULATE & ESTIMATE (NFXP)

```
simulateRust forward path of mileage + replace decisions
function simulateRust(P_rep, θ3, S, T, seed=42)
Random.seed!(seed)
x = zeros(Int,T); d = zeros(Int,T); x[1] = 1
for t in 1:T
    d[t] = rand{< P_rep[x[t]] ? 1 : 0} # Bernoulli
    if t < T
        Δ = rand(Categorical{θ3}) - 1 # 0,1,2
        x[t+1] = d[t]==1 ? min(1+Δ,S) : min(x[t]+Δ,S)
    end
    # replace resets to 1
end
return x, d
end
```

Step 1: estimate θ_3 count observed mileage increments

```
counts = zeros(3)
for t in 1:T-1
    if d[t]==0 && x[t]<S-2
        Δ = x[t+1] - x[t]
    elseif d[t]==1
        Δ = x[t+1] - 1 # reset baseline
    end
    counts[Δ+1] += 1
end
θ3_hat = counts ./ sum(counts)
```

Step 2: NFXP objective *neg log-lik; solves DP at each guess*

```
function nfxpObjective(params, x_d, d_d, β, θ3_hat, S)
RC_e, θ1_e = params
if RC_e<0 || θ1_e<0; return 1e10; end # bounds
m_t = RustModel(β=S, S=S, RC=RC_e, θ1=θ1_e, θ3=θ3_hat)
sol = solveRust(m_t) # INNER loop: VFI
ll = 0.0
for t in eachindex(d_d)
    s = x_d[t]
    ll += d_d[t]*log(max(sol.P[s],1e-15)) +
        (1-d_d[t])*log(max(1-sol.P[s],1e-15))
end
return -ll # minimize => negate
end
```

Run NFXP *outer loop: minimize over (RC,θ1)*

```
res = optimize(
    p -> nfxpObjective(p, x, d, β, θ3_hat, S),
    [15.0, 0.03], WelderMead(),
    Optim.Options(iterations=5000))
RC_hat, θ1_hat = Optim.minimizer(res)
```

NFXP = 2 nested loops: outer ML over (RC, θ_1) , inner VFI solves the DP at every guess. θ_3 estimated *once* up front (Step 1), held fixed. **Identification:** only the gap $v^{rep} - v^{keep}$ is identified $\Rightarrow$ need to normalize (fix one level).

VARIANTS — quadratic & misspec.

VanModel *HW: quadratic cost $c = \theta_2 \cdot \bar{x}^2$*

```
@kwdef struct VanModel
    β::Float64=0.95; S::Int=50; RC::Float64=20.0
    θ2::Float64=0.001078 # quadratic coef
    x::Vector{Float64}=[0.36,0.48,0.16]
    c::Vector{Float64}=Float64{0.0:S-1}
    # only change
end
```

iterateBellman/solveBellman/solveVan are **identical** to Rust; only c changes. Quadratic cost is flatter at low mileage and steeper at high mileage. OLS-thru-origin θ_1^{lin} overstates low/mid states and understates high states.

Mileage-dependent RC *practice final: RC becomes a vector*

```
# RC now length-S: RC(x)=RC0 + θ2 ∙ x[x]
function iterateBellman(n::RustModel, V, F0, F1)
    (; β, RC, c) = m
    v_keep = -c .+ β .*(F0 ∙ V)
    v_rep = -RC + β .*(F1 ∙ V) # ./- broadcasts
    vmax = max.(v_keep, v_rep)
    return vmax .+ log.(exp.(v_keep - vmax))
    .+ exp.(v_rep - vmax))
end
```

now both v^{keep} and v^{rep} slope down in x (was: only v^{keep} , v^{rep} flat). Continuation $\beta F_1 V$ still one scalar (reset). $P(rep)$ effect **ambiguous**: direct flow $(-RC(x))$ pushes down at high x ; feedback (keeping marches toward dear states) pushes up.

Misspecification (NFXP) *fit WRONG cost form to true-quadratic data*

```
# data from VanModel (quadratic); fit RustModel
k (linear) = nfxpObjectiveLin builds RustModel
# at each (RC,θ1), else identical to nfxpObjective
m_1 = RustModel(RC=RC_1, θ1=θ1_1, θ3=θ3_hat)
sol_1 = solveRust(m_1) # implied P
```

linear fit matches P at low/mid mileage (where most decisions are) but **wrong shape** at high mileage (linear understates true cost). Estimator inflates RC, θ_1 to compensate. **Counterfactual** (e.g. $RC \times 0.7$): correctly-specified curve tracks truth; misspecified curve has wrong shape $\Rightarrow$ reduced-form/misspecified structural fails out-of-sample.

=== KYDLAND-PRESCOTT RBC ===
EQUATIONS — for spot-the-error

Resources $R(k) = c_t + k_{t+1} = zk^\alpha + (1 - \delta)k$; $c = R(k) - k'$.

Bellman (consume $R - k'$, discount future):

$$V_t(k, z) = \max\{\log(zk^\alpha + (1-\delta)k - k') + \beta E[V(k', z')|z]\}$$

Last period (no future, eat everything): $V_T(k) = \log(zk^\alpha + (1 - \delta)k)$, $k'=0$.

Euler (interior): $\frac{1}{c_t} = \beta \frac{1}{c_{t+1}} (\alpha zk^{\alpha-1} + 1 - \delta)$.

Det. steady state: $k_{ss} = \left(\frac{\alpha \beta}{1 - \beta(1 - \delta)} \right)^{1/(1 - \alpha)}$.

common errors: missing β on continuation; consumption omits the savings term (must subtract next-period capital); resources written $k^\alpha - \delta k$ instead of $zk^\alpha + (1 - \delta)k$; passing a raw value *vector* to optimal_policy instead of a linear_interpolation callable.

FINITE HORIZON (practice final)

iterateBellman *one backward step on the grid; wrap V_next as interpolant*

```
function iterateBellman(m, V_next)
    (; α, δ, k_grid) = m; z = 1.0
    V = similar(k_grid); kp = similar(k_grid)
    vf = linear_interpolation(k_grid, V_next) # callable!
    for (i,k) in enumerate(k_grid)
        R = zk^α + (1-δ)*k # resources (NOT k^α-δk)
        res = optimal_policy(m, R, vf) # max log(R-k')+βvf(k')
        V[i] = res.V; kp[i] = res.kp
    end
    return (V=v, kp=kp)
end
```

Capital path V_3 *terminal, back-induct, iterate policies forward*

```
z = 1.0
V3 = log.(z .* m.k_grid.^m.α .+ (1-m.δ).*m.k_grid)
res2 = iterateBellman(m, V3) # period-2 policy
res1 = iterateBellman(m, res2.V) # period-1 policy
kp1 = linear_interpolation(m.k_grid, res1.kp)
kp2 = linear_interpolation(m.k_grid, res2.kp)
k2 = kp1(k1); k3 = kp2(k2); k4 = 0.0 # forward
k_path = [k1, k2, k3, k4]
```

V3: last period, no continuation $\Rightarrow$ eat all resources, save nothing. Back-induct $V_3 \rightarrow V_2 \rightarrow V_1$ (policies), then push k_1 forward through the saved kp interpolants.

INFINITE HORIZON (stochastic, lecture)

```
RBCModel + setup! grid + Rouwenhorst-discretized TFP

@kwdef mutable struct RBCModel
    β::0.96; α::0.33; δ::0.10; ρ::0.90; σ_e::0.02
    N::7; N_z=200
    z_grid::Float64[]; P_z=zeros(0,0); k_grid::Float64[]
end
function setup!(m) # fill grids
    (; β,δ,α,k_grid,N,z,N_k) = m
    lz, P = rouwenhorst(N,z, P, α, σ_e)
    m.z_grid = exp.(lz); m.P_z = P
    k_ss = (α*β/(1-β*(1-δ)))^(1/(1-α))
    m.k_grid = collect(LinRange(0.2k_ss, 1.8k_ss, N_k))
end
```

optimal_policy *maximize over k' at one state (k,iz)*

```
function optimal_policy(m, k, iz, V_itp)
    (; β,α,δ,z_grid,P,z,N,z,k_grid) = m
    z = z_grid[iz]
    budget = z*k^α + (1-δ)*k # resources
    function obj(kp)
        c = budget - kp
        c <= 0 && return 1e10 # infeasible penalty
    end
```

```
EV = sum(P_z[iz,jz]*V_itp[jz](kp) for jz in 1:N_z)
return -(log(c) + β*EV) # negate: minimize
end
k_max = min(k_grid[end], budget-1e-8)
res = optimize(obj, k_grid[1], k_max, Brent())
return (V=Optim.minimum(res), kp=Optim.minimizer(res),
        c=budget-Optim.minimizer(res))
```

end

bellman_operator + solveRBC *loop all (k,z); VFI to fixed point*

```
function bellman_operator(m, V)
    (; N,k,N_z,k_grid) = m
    V_new=similar(V); kp=similar(V); cp=similar(V)
    V_itp = [linear_interpolation(k_grid, V[:,jz])
             for jz in 1:N_z] # wrap each z-column
    for iz in 1:N_z, ik in 1:N_k
        r = optimal_policy(m, k_grid[ik], iz, V_itp)
        V_new[ik,iz]=r.V; kp[ik,iz]=r.kp; cp[ik,iz]=r.c
    end
    return V_new, kp, cp
end
function solveRBC(m::RBCModel;
    tol=1e-6, maxiter=1000, verbose=true)
    (; α, δ, z_grid, k_grid, N_k, N_z) = m
    V = zeros(N_k, N_z) # initial guess: V = 0
    V_new, kp_policy, c_policy = bellman_operator(m, V)
    #iterate the Bellman operator until convergence
    dist = 1.0; iter = 0
    while dist > tol && iter < maxiter
        V_new, kp_policy, c_policy = bellman_operator(m, V)
        dist = maximum(abs.(V_new - V))
        V = V_new
        iter += 1
    end
    if dist < tol && verbose
        println("Converged in $iter iterations."*
            "(dist = $(round(dist, digits=3)))")
    end
    #Interpolate the policy functions
    kp_itp = [linear_interpolation(k_grid, kp_policy[:, iz])
             for iz in 1:N_z]
    c_itp = [linear_interpolation(k_grid, c_policy[:, iz])
             for iz in 1:N_z]
    #Return the solution
    return (V=v, kp=kp_policy, c=c_policy,
            kp_itp=kp_itp, c_itp=c_itp)
end
```

2D state (k, z) : z via Rouwenhorst Markov chain, k continuous $\Rightarrow$ interpolate V off grid. $\rho \uparrow$: shocks persist, bigger swings. $\beta \uparrow$: more saving, higher k^* .

steady_state(m) *closed-form deterministic SS*

```
function steady_state(m)
    (; α, β, δ) = m
    k = (α*β/(1-β*(1-δ)))^(1/(1-α))
    y = k^α; c = y - δ*k; i = δ*k
    return (k=k, y=y, c=c, i=i, r=α*k^(1-α))
end
```

rouwenhorst(N,ρ,σ_ε) *discretize AR(1) TFP $\rightarrow$ grid + Markov P*

```
function rouwenhorst(N, ρ, σ_e)
    σ_z = σ_e / sqrt(1-ρ^2); p = (1+ρ)/2
    P = [p 1-p; 1-p p] # N=2 base
    for n in 3:N
        z = zeros(n-1)
        P = p*[P; z', 0] + (1-p)*[z P; 0 z'] +
            (1-p)*[z', 0; P z] + p*[0 z; 1] #
        p[2:end-1, :] ./= 2 # renormalize interior
    end
    zg = collect(LinRange(-sqrt(N-1)*σ_z, sqrt(N-1)*σ_z, N))
    return zg, P
end
```

simulateRBC *forward sim; z via Markov, k via policy interpolant*

```
function simulateRBC(m, sol, T; seed=42)
Random.seed!(seed); (; α,δ,z_grid,P,z,N,z) = m
k=zeros(T); z=zeros(T); z1=zeros(Int,T)
y=zeros(T); c=zeros(T); i=zeros(T)
k[1]=steady_state(m).k; z1[1]=(N_z+1)/2 # median z
for t in 1:T
    z[t]=z_grid[z1[t]]
    kp = sol.kp_itp[z1[t]](k[t]) # interp policy
    y[t]=z[t]*k[t]^α
    c[t]=y[t]+(1-δ)*k[t]-kp; i[t]=kp-(1-δ)*k[t]
    if t<T
        k[t+1]=kp
        z1[t+1]=rand(Categorical(P,z[z1[t],:])) # next z
    end
end
return DataFrame(t=1:T,k=k,z=z,y=y,c=c,i=i)
end
```

Business-cycle moments *volatilities relative to output, comovement*

```
σ_y=std(sim.y_dev); σ_c=std(sim.c_dev) # % devs
σ_k=std(sim.i_dev)
# σ relative to output:
[1.0, σ_c/σ_y, y1/σ_y] # C smoother, I ~3* as volatile
# comovement with y:
[cor(sim.y_dev,sim.c_dev), cor(sim.y_dev,sim.i_dev)]
```

key RBC facts: $\sigma_c < \sigma_y < \sigma_i$ (consumption smoothing; investment $3 \times$ output vol); c, i strongly procyclical (cor $w/y \approx 1$); shocks propagate via capital accumulation.

LINEARIZATION (perturbation)

Euler: $1/c_t = \beta(1/c_{t+1})(\alpha z k^{\alpha-1} + 1 - \delta)$. Linearize Euler + resource constraint around SS, guess a *linear* policy, match coefficients $\Rightarrow (\phi_k, \phi_z)$. Fast & accurate *near* SS; VFI is global (handles large deviations, binding constraints). Curves overlap near k^* , diverge in the tails.

=== ROOT-SOLVE · OPTIM · JULIA ===

NonlinearSolve — f(u,p)=0 *3 rules: f takes (u,p); returns a VECTOR; u0 a VECTOR*

```
using NonlinearSolve
f(u,p) = [u[1]^2 - p.target] # vector output!
prob = NonlinearProblem(f, [1.0], (target=4.0,))
sol = solve(prob); sol.u[1] # 2.0
# no params: f(u,_)=[u[1]^3-2]; NonlinearProblem(f,[1.0])
# solve(prob)=Newton; solve(prob,TrustRegion()) robust
```

Market equilibrium *excess demand $D-S=0$*

```
function excess_demand(u, p)
    price = u[1]
    D = p.*price.^(-p.epsd); S = p.*price^(p.epas)
    return [D - S] # vector!
end
par = (A=100., epsd=1.5, B=80., epss=1.)
sol = solve(NonlinearProblem(excess_demand,[1.0],par))
p_star = sol.u[1]
```

Optim — MINIMIZES *to maximize: negate inside, negate result outside*

```
using Optim
res = optimize(x -> f(x,p), lo, hi) # scalar bracket
res = optimize(x -> f(x,p), x0) # x0 VECTOR
res = optimize(x -> f(x,p), x0, LBFGS()) # w/ gradient
Optim.minimizer(res) # argmin
Optim.minimum(res) # f(argmin); negate if maximizing
```

Bounded (probabilities) *Fminbox keeps params in [lo,hi]*

```
res = optimize(θ -> neg_ll(θ,s),
    [0.01,0.01], [0.99,0.99], [0.5,0.5], # lo,hi,x0
    Fminbox(LBFGS()))
# without Fminbox: solver wanders to p<0, log(0) blows up
```

MLE pattern *minimize NEGATIVE log-likelihood; guard bad params*

```
function neg_ll(v, data)
    μ, σ = v[1], v[2]
    σ <= 0 && return Inf # guard
    N = length(data)
    ll = -N*log(σ*sqrt(2π)) - sum((data.-μ).^2)/(2σ^2)
    return -ll # FLIP for minimization
end
res = optimize(v -> neg_ll(v,data), [0.,1.])
```

Linear algebra essentials *solve, verify, multipliers*

```
z = A \ b # SOLVE Ax=b (QR; never inv!)
det(A) # !=0 => invertible
L = inv(I - A) # Leontief inverse
sum(L, dims=1) # column = output multipliers
ρ = maximum(abs.(eigvals(A))) # spectral radius <1
```

OLS by hand $\beta = inv(X'X) X'y$; use $X|y$ for stability

```
X = hcat(ones(n), X_raw) # add intercept column
β = X \ y # QR, stable
res = y - X*β; k = size(X,2)
σ² = dot(res,res)/(n-k)
se = sqrt.(diag(σ²' \ inv(X'X))) # std errors
t = β ./ se # |t|>1.96 => reject β=0
```

Markov chain essentials *rows sum to 1; $P^k \cdot \bar{x} = k$ -step E; π^* steady state*

```
P = [0.97 0.03; 0.40 0.60]; sum(P,dims=2) = 1
P * k' = x' # E[X_t|kstate i] (row i)
V = (I - β*P) \ x # PDV: ∑ β^k P^k x closed form
ev = eigen(P'); π = ev.vectors[:,argmin(abs.(ev.values.-1))]
π = real(π ./ sum(π)) # steady state (left eigvec, λ=1)
```

Plots — basics *plot/plot!/scatter; labels via xlabel!/title!*

```
using Plots
plot(x, y, linewidth=2, label="V(k)",
    legend=:bottomright, size=(700,400))
plot!(x, y2, linestyle=:dash, label="2nd series") # overlay
scatter!(Xg, yg, markersize=6, label="grid pts")
xlabel!("W(k)", title="Value fn")
vline!([k_ss], color=:gray, linestyle=:dash, label="k_ss")
hline!([δ*k_ss], color=:gray, label="ss invest")
```